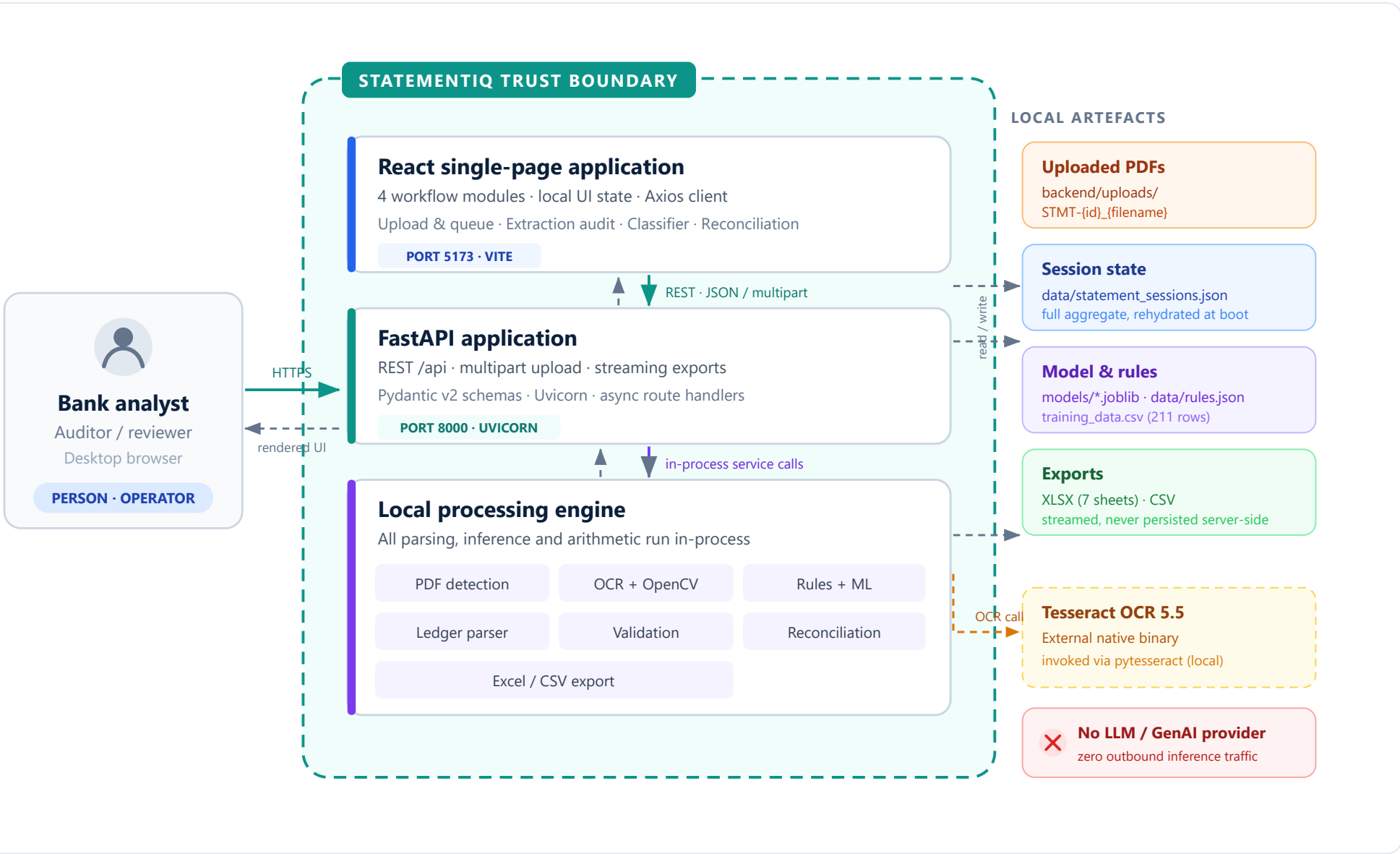
01 System context

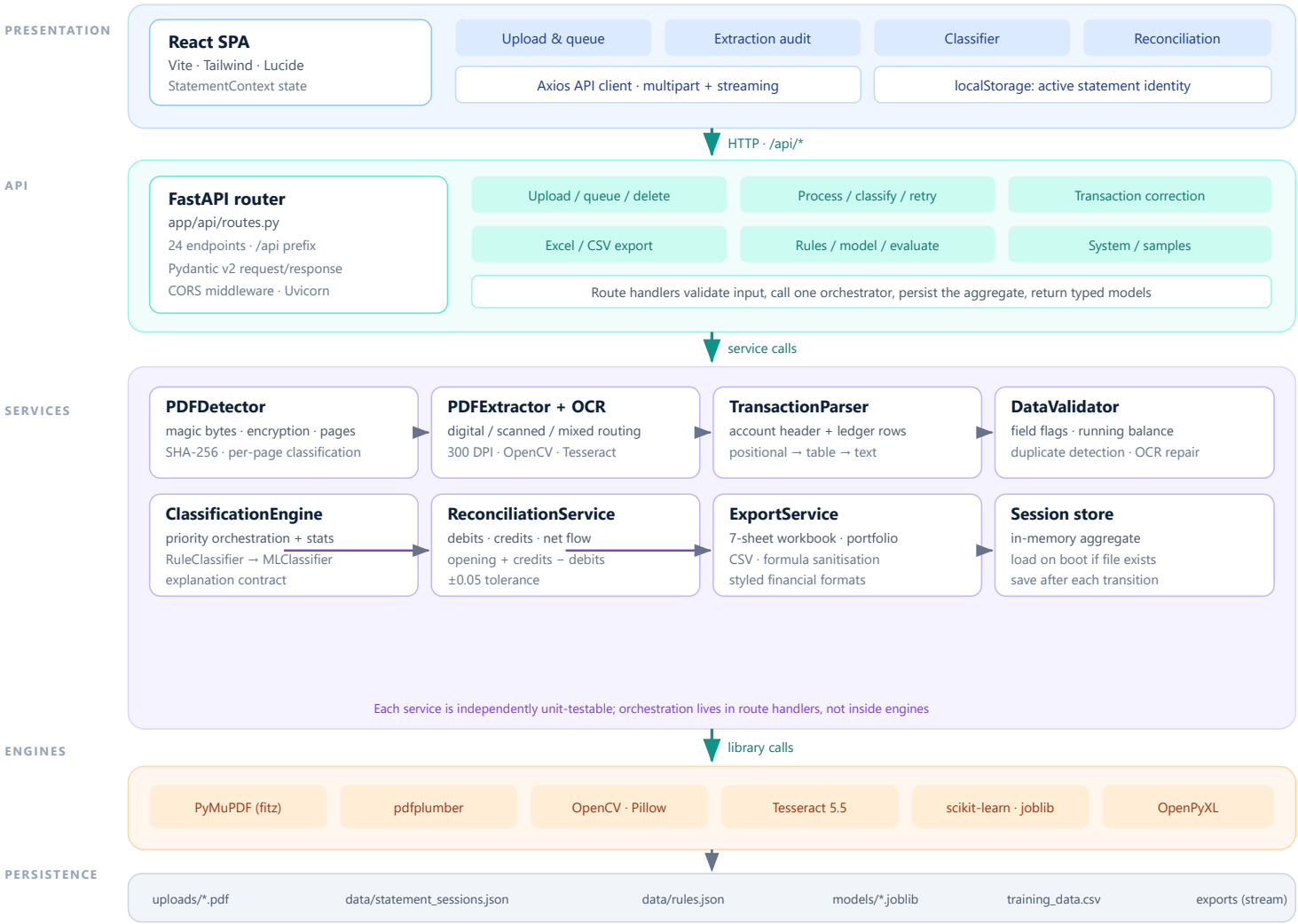
Who talks to the system, what crosses the trust boundary, and what is deliberately absent.



The trust boundary contains every processing step. No statement content leaves the host for inference.

02 Container & component architecture

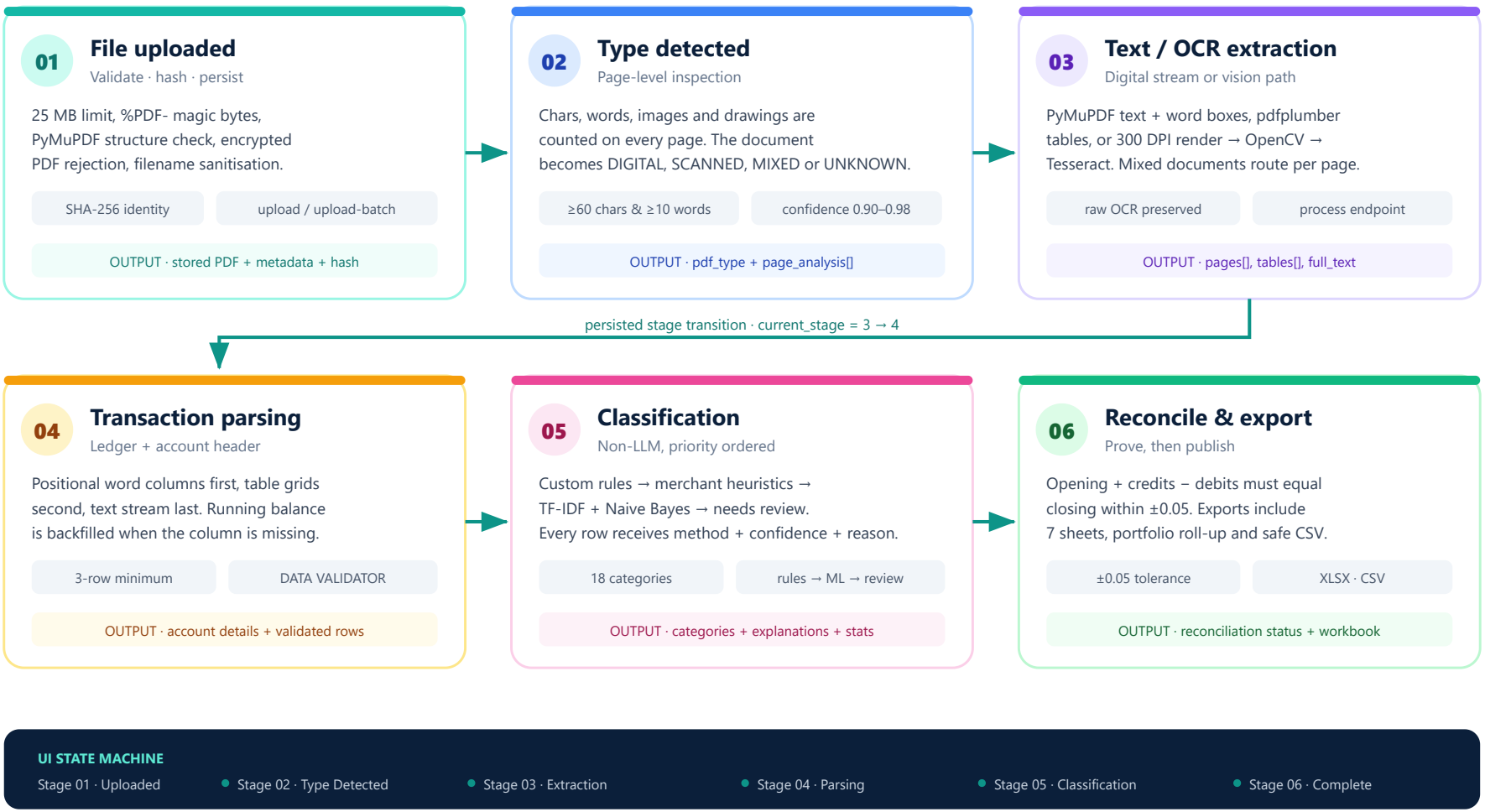
Five layers and the path of a single request from the React UI to a persisted statement aggregate.



- The API layer is deliberately thin. Domain services own the financial logic and remain independently testable.

03 End-to-end processing pipeline

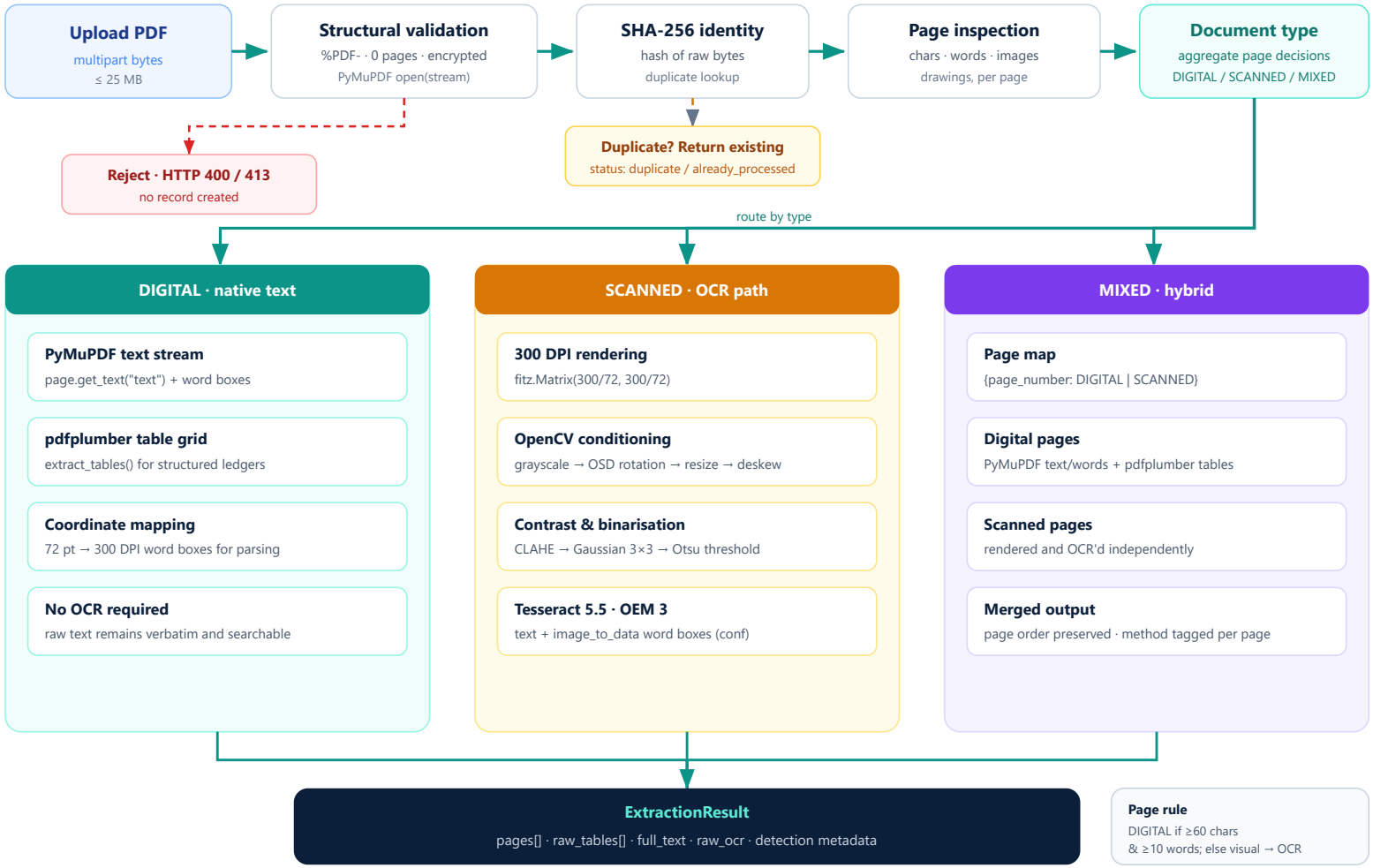
Six stages from upload to reconciled workbook, with the committed artefact at every transition.



- Stage transitions persist to statement_sessions.json; a reload reconstructs the same pipeline state.

04 PDF detection & OCR routing

Per-page evidence decides DIGITAL, SCANNED or MIXED; scanned pages take the OpenCV + Tesseract path.



- A document is not trusted by extension. Magic bytes, page structure and per-page text density are checked.

05 Parsing & ledger validation

Three extraction strategies, amount/date normalisation, anomaly flags and evidence-backed OCR repair.

1 · EXTRACT

ACCOUNT HEADER

Bank name · account holder
Account number → XXXXXX••••
IFSC / routing code
Statement period · currency
Opening balance · closing balance
Fallback: known-bank list + labelled fields

LEDGER ROWS

Date · Description / narration
Debit · Credit · Running balance
Multi-line narration joined
Dr / Cr suffix on single amount column
Page continuity across breaks

DATE | NARRATION | DEBIT | CREDIT | BALANCE

STRATEGY FALLBACK

positional | table grid | text stream

Use a strategy only if it yields ≥ 3 rows; otherwise fall through.
If every balance is missing, backfill from opening balance.

2 · NORMALISE

AMOUNT NORMALISATION

Indian grouping: 1,20,000.00
International: 1,234.56
OCR dots: 1.65.200.00 → 165200.00
Trailing Dr / Cr markers
Accounting negatives: (500.00)
Currency symbols: ₹ / INR / Rs.
Nil, dash, empty → 0.00
Pattern fallback extracts first valid float

DATE NORMALISATION

19 accepted formats → ISO YYYY-MM-DD
DD/MM/YYYY · DD-MM-YYYY · DD.MM.YY
DD Mon YYYY · Mon DD, YYYY · ISO
Sanity window: 1990 – 2040
Original date string retained for audit

RUNNING BALANCE BACKFILL

If no row has a balance but opening balance is known, compute:

running += credit − debit

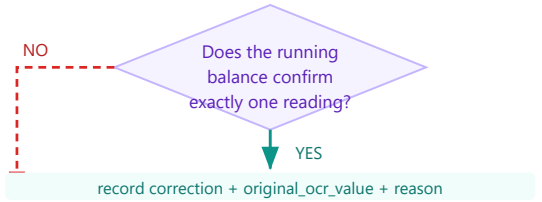
Each computed balance is rounded to 2 decimals.

3 · VALIDATE

ANOMALY FLAGS

missing_date · non_standard_date
empty_or_vague_description
zero_amount · negative_debit / credit
simultaneous_debit_credit
balance_mismatch · overdraft
exact / potential duplicate transaction

OCR CORRECTION GATE



IF AMBIGUOUS

Do not modify the numbers.
Set validation flag uncertain_ocr_amount.
Set review_status = Needs Review.
Surface the row in the Review Flags sheet.
A human decides; the audit trail stays intact.
Never guess a debit or credit.

PROOF OBLIGATIONS

B(prev) + Credit − Debit = B(curr)
row-level running balance · tolerance ±0.05

Opening + ΣCredits − ΣDebits = Closing
statement-level reconciliation · tolerance ±0.05

discrepancy = calc − reported
surfaced, never hidden or rounded away

- Ambiguous OCR is escalated. Corrections happen only when the running balance proves exactly one reading.

06 Non-LLM classification engine

Custom rules → merchant heuristics → TF-IDF + Naive Bayes → human review. No generative model.

1 Custom user rules

confidence 1.00 · method "Custom Rule"

Keywords registered from the dashboard. Absolute precedence over every default rule; ideal for landlord names, internal transfers and rare merchants.

EXACT KEYWORD MATCH

priority field · case option

2 Predefined heuristics

confidence 0.90 – 0.98 · method "Rule-Based"

High-specificity merchants (SWIGGY, ZOMATO, BESCOM, AMAZON, UBER, APOLLO, SALARY CREDIT) are evaluated before generic rails (UPI, NEFT, IMPS, TRANSFER). Word boundaries prevent substring false positives.

18 CATEGORIES

merchant → category map

JSON overlay: data/rules.json

3 Supervised machine learning

TF-IDF (1–2 grams) + Multinomial Naive Bayes (α = 0.2)

Description is lowercased, punctuation stripped, then vectorised with sublinear TF and unigrams + bigrams. The model returns the top-3 class probabilities. It is evaluated only when tiers 1 and 2 produce no rule.

MODEL ARTEFACTS

211 training records

joblib · local inference

p ≥ 0.45 → assign predicted category

p ≥ 0.70 → Verified · 0.45–0.70 → Review

4 Fallback · needs review

category "Others" · method "ML" or "Unclassified"

No rule fired and the top probability is below 0.45. The transaction is kept, labelled Others, and pushed to the review queue with its candidate probabilities intact. Manual override remains preserved on re-runs.

OUTPUT CONTRACT · EVERY ROW

```
{
  "category": "Food & Dining",
  "classification_method": "Rule-Based",
  "confidence": 0.98,
  "review_status": "Verified",
  "explanation": {
    "matched_keyword": "SWIGGY",
    "reason": "Matched rule keyword",
    "model_probabilities": []
  }
}
```

ML rows replace the keyword with top-3 class probabilities.

No external inference call.

CATEGORY TAXONOMY

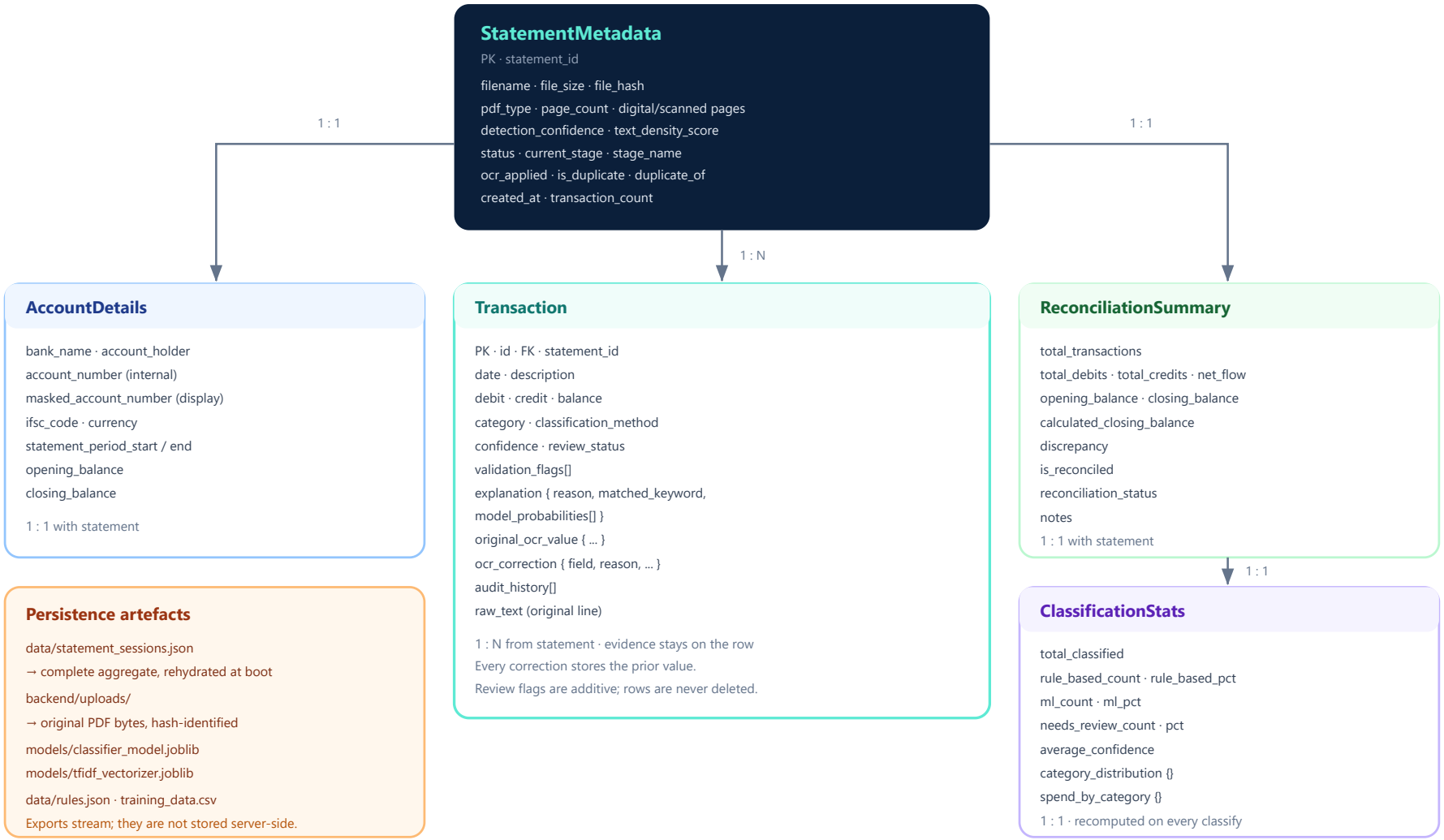
Salary · Food & Dining · Shopping · Travel & Transport · Utilities & Bills · Rent · Healthcare · Education · EMI/Loan · ATM Withdrawal · Bank Charges · Interest · Refunds · Transfers · Other Income · Others · Investments · Other Expenses

Deterministic rules handle recurring spend;
ML is a fallback, never the first word.

- Rules encode recurring Indian banking behaviour; the model is a fallback for narrations the rules miss.

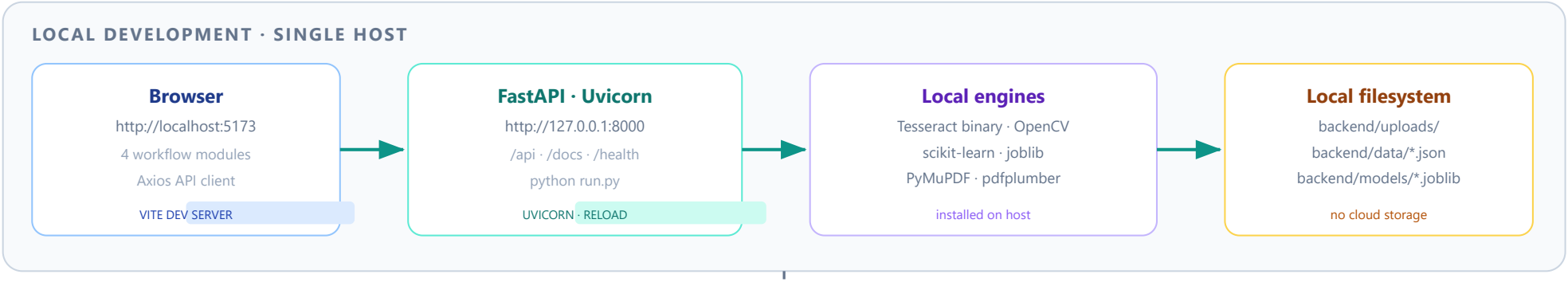
07 Data model

The statement aggregate: metadata, account, transactions, reconciliation, statistics and evidence.



08 Deployment topology

Local development on one host; public demo as a static Vercel SPA plus a Render Docker API.



git push · CI build

VERCEL · STATIC FRONTEND

Build: `npm run build` → `dist/`

Framework preset: Vite · root: frontend

env: `VITE_API_URL` → Render API URL

rewrites: SPA fallback in vercel.json

output: static assets + index.html served at edge

Request path

Browser → Vercel edge (HTML/JS/CSS)

→ VITE_API_URL → Render FastAPI container

No API proxy on Vercel; the SPA calls the API directly.

vercel.json	Vite 5.4	React 18
SPA rewrite	production bundle	4 modules · Tailwind

RENDER · DOCKER BACKEND

Base: `python:3.11-slim`

OS packages: tesseract-ocr · libtesseract-dev · libgl1 · libglib2.0-0

Python: requirements.txt (FastAPI, PyMuPDF, OpenCV, scikit-learn, OpenPyXL)

Start: `python backend/run.py` · port `8000`

Health: `GET /health` → {"status":"healthy"}

Runtime caveat

Render free instances use an ephemeral filesystem. Uploads and session JSON survive only for the instance lifetime.

Attach a disk or external DB for production.

Environment

PORT=8000

TESSERACT_CMD=...

VITE_API_URL=...

Tesseract auto-discovered on PATH.

Container isolation makes OCR/OpenCV reproducible across hosts.

- Tesseract and OpenCV are OS-level dependencies, which is why the backend ships as a container.